

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
df = pd.read_csv("/content/sample_data/archive (5).zip")

print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:")
print(df.head())
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDataset Info:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())

plt.figure(figsize=(10,8))
sns.heatmap(df.drop('label', axis=1).corr(),
            annot=True,
            cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

X = df.drop("label", axis=1)
y = df["label"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print("\nModel Accuracy:")
print(round(accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(y_test, predictions))

importance = model.feature_importances_

feature_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

feature_df = feature_df.sort_values(
    by='Importance',
    ascending=False
)

print("\nFeature Importance:")
print(feature_df)

plt.figure(figsize=(8,5))
sns.barplot(

```

```
x='Importance',
y='Feature',
data=feature_df
)
plt.title("Feature Importance")
plt.show()
print("\n===== Crop Prediction =====\n")

N = float(input("Enter Nitrogen (N): "))
P = float(input("Enter Phosphorus (P): "))
K = float(input("Enter Potassium (K): "))
temperature = float(input("Enter Temperature: "))
humidity = float(input("Enter Humidity: "))
ph = float(input("Enter pH Value: "))
rainfall = float(input("Enter Rainfall: "))

user_data = np.array([[
    N,
    P,
    K,
    temperature,
    humidity,
    ph,
    rainfall
]])

prediction = model.predict(user_data)

print("\nRecommended Crop:")
print(prediction[0])
```

